

Logic/Pseudocode for Individual Project
Dominic Kroll
ITDEV117_900
SP2026 MATC

Start

```
// initialize classes
```

```
class Envelope
```

```
  private attack = 0.1
```

```
  public setAttack(val)  
    attack = val
```

```
  public getAttack()  
    return attack
```

```
  private decay = 0.2
```

```
  public setDecay(val)  
    decay = val
```

```
  public getDecay()  
    return decay
```

```
  private sustain = 0.8
```

```
  public setSustain(val)  
    sustain = val
```

```
  public getSustain()  
    return sustain
```

```
  private release = 0.5
```

```
  public setRelease(val)  
    release = val
```

```
  public getRelease()  
    return release
```

```
endClass
```

```
class Filter
```

```
  private cutoff = 1000, resonance = 0.3, type = "LowPass"
```

```

    public setCutoff(val)
        cutoff = val

    public getCutoff()
        return cutoff
endClass

class SamplerEngine
    private activeSample = null
    private pitchOffset = 0.0

    public load_Sample(file)
        activeSample = load_file(file)

    public getSample()
        return activeSample

    public setPitch(val)
        pitchOffset = val
endClass

class WavetableEngine
    private position = 0.0

    public setPosition(val)
        position = val

    public getPosition()
        return position
endClass

class FXRack
    private FXlist = ["Distortion", "Filter", "EQ", "Reverb", "Delay"]

    public function addEffect(name):
        fx_list.append(name)

    public function removeEffect(index):
        fx_list.remove_at(index)

    public getFXList()
        return FXlist
endClass

```

```

// Instances of Classes
object myEnvelope = new Envelope()
object myFilter = new Filter()
object mySampler = new SamplerEngine()
object myWavetable = new WavetableEngine()
object myFX = new FXRack()

// object declarations

// Audio Data Objects/Buffers
object activeSample = null // Holds the audio data for the loaded sample
object audioBus // Final connection for output to DAW
buffer audioSource // Raw digital audio stream to be manipulated

// Sound Engine Module Objects
object LFOMod = {rate: 1.5, wave: "Sine", depth: 0.5}

// UI containers
object MainUIFrame // The physical window of the VST
object samplerView // The screen for samples
object waveTableView // The screen for the synth

// variable declarations

boolean vstOpen = true
String currentMode = "NONE" // Tracks if we are in Wavetable or Sampler mode
String selectedCategory = "NONE" // Stores the user's menu choice
boolean midiKeyHeld = FALSE // True when a user is pressing a key
float waveTablePosition = 0.0 // The specific point in the wavetable
float pitchOffset = 0.0 // Adjustment for sample pitch/octave
list effectsRack = ["Distortion", "Filter", "EQ", "Reverb", "Delay", "Phaser"]
list mainMenu = ["Live Instruments/Samples", "Wavetable Generator"]
list subMenu = ["Drums", "Bass", "Keys", "Pads", "Leads", "FX"]
list subMenuDrums = ["Kicks", "Snares/Rimshots", "Hats", "Percs", "Shakers", "Other"]
list subMenuBass = ["808s", "Reeses/Sustains", "Bass Plucks", "Growls", "Wobbles",
"Other"]
list subMenuKeys = ["Piano", "Keyboard", "Synth Keys"]
list subMenuPads = ["Ambient", "Evolving", "Dark", "Grainy"]
list subMenuLeads = ["Trap Leads", "Dark Leads", "Ambient Leads", "Classic"]
list subMenuFx = ["Grainy Textures", "Atmospheric", "Glitch", "Vocoded/Downsampled",
"FoleyFx"]

```

```

// Main Program Loop
while vstOpen == TRUE

    // First step ( Dropdown Menu Logic)
    display mainMenu

    if user_selection == "Wavetable Generator":
        set currentMode = "WAVETABLE"
        display wavetable_ui (Oscillator, Position, WaveType)
        enable parameters (ADSR, LFO_1, Filter_1)

    else if userSelection == "Live Instruments/Samples":
        display subMenu
        set selectedCategory = user_input

    // Final selection branch
    display sampleList(selected_category)
    if user_selects_sample
        set activeSample = load file(user_selected_file)
        set currentMode = "SAMPLER"
        display sampler_ui (sample_screen, waveform_display)
        enable parameters (Pitch, Reverse, Clip_Start/End)

    // Signaflow (Handling the sound)
    // This part triggers when the system detects MIDI input
    while midiKeyHeld == TRUE:
        if currentMode == "WAVETABLE":
            audioSource = myWavetable.getPosition()
        else if currentMode == "SAMPLER":
            audioSource = PLAY_SAMPLE_FILE(active_sample, pitch_offset)

        // Envelope & Modulation (Applied to either source)
        audioSource = apply_Envelope(audioSource, myEnvelope.getAttack())
        audio_source = apply_LFO(audioSource, LFOMod)
        audioSource = apply_Filter(audioSource, myFilter.getCutoff())

        // Effects Rack (The end of the chain)
        // Passes the audio through the list of effects
        finalOutput = apply effectsRack(audioSource, myFX.getFXList())

        output finalOutput to audioBus
    end while

// Exit Logic

```

```
// Clean up and exit if user hits close program
if user_clicks_close_button:
    set vstOpen = FALSE
    call Cleanup_Memory()

end while

End
```